ASSIGNMENT 3

Database Design Analysis: Functional Dependencies, Anomalies and Improvements

Project: Library Management System

Database: MySQL

Tables in finalized design: Authors, Publishers, Categories, Books, Members, Staff, Loans, Reservations, Fines

Note: Replace the student name, course details, and screenshots with your own details before submission. The query outputs in this report are illustrative outputs based on the sample records shown.

1. Introduction

The Library Management System stores information about books, authors, publishers, categories, members, staff, loans, reservations and fines. The design separates major entities into nine tables and connects related records through primary and foreign keys. This reduces repeated descriptive data and helps maintain relationships.

2. Functional dependencies

A functional dependency X → Y means that a value of attribute set X uniquely determines a value of attribute set Y. The following dependencies are expected in the proposed schema:

Table	Functional dependencies
Authors	AuthorID → AuthorName, Nationality, BirthYear
Publishers	PublisherID → PublisherName, Address, Phone
Categories	CategoryID → CategoryName, Description; CategoryName should ideally be UNIQUE
Books	BookID → Title, ISBN, AuthorID, PublisherID, CategoryID, PublicationYear, TotalCopies, AvailableCopies; ISBN → BookID and other book attributes when ISBN is present and unique
Members	MemberID → FullName, Email, Phone, Address, MembershipDate, MembershipType; Email → MemberID and other member attributes when Email is not NULL
Staff	StaffID → FullName, Role, Email, Phone, HireDate
Loans	LoanID → BookID, MemberID, StaffID, IssueDate, DueDate, ReturnDate, Status
Reservations	ReservationID → BookID, MemberID, ReservationDate, Status
Fines	FineID → LoanID, Amount, PaidStatus, PaidDate; LoanID → FineID and fine attributes because LoanID is UNIQUE

Foreign keys such as Books.AuthorID → Authors.AuthorID express referential relationships; they are not, by themselves, functional dependencies in the same sense as a table's key determining its non-key attributes.

3. Data redundancy and anomalies

Data redundancy occurs when the same fact is stored repeatedly. In a poorly designed single table, every loan row might repeat the book title, author name, publisher address, member contact details and staff name. Updating a publisher's address would then require editing many rows. The nine-table design already reduces much of this redundancy by storing each entity's details once.

Anomaly	Example in a poorly designed combined table	How the proposed design helps
Insertion	A new author cannot be recorded until at least one of their books is added, if author details exist only in a loan/book transaction row.	Authors is independent, so an author can be added before a book or loan.

Anomaly	Example in a poorly designed combined table	How the proposed design helps
Update	Changing a member's phone number requires changing every loan row containing that member.	Member contact data is stored once in Members.
Deletion	Deleting the only loan row for a book could accidentally remove the only stored copy of its title, author and publisher details.	Books and related descriptive entities exist independently of Loans.
Repeated facts	Author, publisher and category names are repeated for every loan of a book.	Books stores foreign keys to Authors, Publishers and Categories.

4. Potential issues remaining in the current schema

• Books permits NULL foreign keys because AuthorID, PublisherID and CategoryID are not declared NOT NULL. If every book must have these details, make them NOT NULL.
• Categories.CategoryName is not UNIQUE, so duplicate category names could be entered.
• Staff.Email is not UNIQUE, unlike Members.Email. Consider making it UNIQUE if each staff member must have a distinct email.
• TotalCopies and AvailableCopies can accept negative values or AvailableCopies greater than TotalCopies without CHECK constraints.
• MembershipType, loan Status, reservation Status and PaidStatus accept arbitrary text values. CHECK constraints or lookup tables can restrict valid values.
• A single AuthorID in Books supports one author per book only. If books can have multiple authors, use a junction table BookAuthors(BookID, AuthorID).
• AvailableCopies duplicates information that may be derivable from copies and active loans. If it is stored for performance, transactions must update it reliably; alternatively, use a copy-level inventory design.

5. Recommended improvements

Improvement	Suggested implementation / reason
Strengthen domain constraints	Add CHECK (TotalCopies >= 0), CHECK (AvailableCopies >= 0 AND AvailableCopies <= TotalCopies), and CHECK constraints for allowed statuses.
Prevent duplicate category/staff details	Add UNIQUE to Categories.CategoryName and, where appropriate, Staff.Email.
Require essential relationships	Use NOT NULL on Books' foreign keys if each book must have an author, publisher and category.
Support multiple authors	Create BookAuthors(BookID, AuthorID) with a composite primary key if co-authorship is required.
Track physical copies accurately	For larger libraries, create BookCopies(CopyID, BookID, CopyStatus) rather than relying only on aggregate copy counts.
Use transactions	Issue/return a book and adjust inventory within one transaction to avoid inconsistent counts.
Add indexes thoughtfully	Index frequently searched foreign keys and fields such as Loans.MemberID, Loans.BookID, Reservations.MemberID and Books.Title.

6. Conclusion

The nine-table design separates independent entities and reduces common insertion, deletion and update anomalies. Additional NOT NULL, UNIQUE, CHECK and status constraints, together with a junction table if multiple authors are allowed, would improve correctness and make the database more robust.